

Concept2Cure.RI — Investor Technical Brief

Version · 2026-09-08, revised 2026-09-28

Codebase branch · `concept2cure-v2` (the sole production branch)

Audience · technical diligence readers and their principals

PREPARED FOR PROSPECTIVE INVESTORS

Table of Contents

1. Executive summary
2. What the product is, and what it replaces
3. The regulated use cases we address
4. The engineered workstreams in the tree
5. System architecture at a glance
6. The control plane: kernel, decisions, and adaptive policy
7. The AI Gateway and approved-model governance
8. Retrieval: the 9-step provenance-tracked RAG pipeline
9. The intelligence layer: RIM, CORTEX Prime, Precedent Engine
10. The three-layer memory system
11. Submission Twin and the Intelligent Report Engine
12. The authoring surface and governed actions
13. The database: schema, migrations, row-level tenancy
14. Security, identity, and 21 CFR Part 11 compliance
15. The frontend surface, design system, and design governance
16. The harness: our engineering discipline as a moat
17. Validation, testing, observability, and reliability
18. The competitive moat, in one page
19. The launch definition, and where each row stands
20. Appendix — quantitative inventory

1. Executive summary

Concept2Cure.RI is a regulated-industry operating system for life-sciences submission work. It replaces the eCTD authoring silo, the regulatory intelligence silo, the QMS/audit silo, and the shared-drive-plus-Word workflow with a single, governed, memory-carrying environment in which the AI is a first-class participant — but never an unchecked one.

Where competitors sell either an AI chat wrapper over a document store, or a static submission tool with a chatbot bolted on, we have built the missing middle. **Deterministic engines** produce the numbers and verdicts, and the model narrates them. An **AI gateway** serves only approved, version-pinned models. A **governance kernel** returns allow, review or deny per domain. A **three-layer memory** carries what the team has decided across sessions and users.

What ships first is deliberately narrow: six apps — Projects, Vault, Authoring, Submission Center, Submission Readiness and QMS controlled documents. The wider tree, covering device, EU MDR, CSR, post-market and RIM work, stays behind a flag that is off in production until the ten launch rows in §19 are green. The validation package for those six apps has been executed on a clean install in the production posture: operational qualification 101 pass, 0 fail, 2 deviations of 103.

The verified size of what we have built:

METRIC	VALUE
Server TypeScript files	4,180
Client React files (.tsx)	439
HTTP route modules (top level)	368
Service files (recursive)	2,956 across 222 service directories
Database migration files	554 (247 in migrations/, 307 in db/migrations/)
pgTable declarations across schema files	~694
Multi-tenant scoping references (organizationId) in shared/schema.ts alone	736
Automated test files (.test.ts/.test.tsx/.spec.ts)	2,348 total, 448 in tests/, 1,522 under server/
Docs .md files under docs/	672 across 55 subdirectories

Every one of those numbers is grounded in a specific path we can walk a diligence reader to, and §20 gives the exact command behind each. They were measured on 2026-09-08 against `concept2cure-v2`; the repository took 80 commits that day alone, so expect the counts to have grown by the time you re-run them. Re-run them — that is the point of publishing the commands.

2. What the product is, and what it replaces

Concept2Cure.RI is the operating system for regulated submission work at drug, device, and diagnostic companies. It is not a chatbot. It is not a document store. It is an environment in which:

- Every artifact is versioned, provenance-linked, and traceable to source evidence.
- Consequential AI actions are routed through a policy kernel that returns *allow* / *review* / *deny* per domain, with a rationale and a regulatory reference.
- Governed actions are gated by a tiered, fail-closed Part 11 check: record-altering commands require a reason-for-change, and the high-impact tier (freeze, sign, submit) additionally requires re-authentication — password plus TOTP where the signer has MFA enabled.
- Every generation is grounded in a retrieval pass across the tenant's own evidence, and the resulting draft carries sentence-level provenance the reviewer can follow to source.
- Every project accumulates a memory: locked facts, decisions, open questions, next actions, artifacts already produced — reusable across sessions and users without re-briefing the AI.

What it replaces, workstream by workstream:

- **Veeva Vault** / **MasterControl** for authoring and QMS.
- **DXC** / **Certara** / **Lorenz** for eCTD lifecycle and submission gateway management.
- **Cortellis** / **Citeline** as the regulatory intelligence lookup tool, superseded by a stack that reads your project's evidence against the regulator corpus and gives you a next best action rather than a search result.
- The **spreadsheet-and-Word workflow** that most small sponsors still use for pre-IND, pre-sub, and CRL response drafting — collapsed into one governed surface.

The product is used before, during, and after submission. Before: gap register, precedent mining, protocol design. During: co-authored eCTD Modules 1–5, section-level readiness, reviewer-simulation adversarial review. After: PSUR/DSUR periodic reporting, CAPA, deficiency response, RIM registration and label lifecycle. The launch release covers the middle of that arc — authoring, eCTD assembly, submission readiness and QMS controlled

documents — and so replaces the first two rows above. Intelligence and post-market work is in the tree, behind the flag.

3. The regulated use cases we address

The tree covers nine regulated use cases. Each is built as a workstream with its own routes, services and schema, and each is a distinct commercial anchor. Not all nine are in the launch release. Since 2026-09-25 the API refuses surfaces outside it in production (`server/services/entitlements/launch-scope-api.ts`), and since 2026-09-26 so do AnA's in-process tools (`server/services/ana/ana-launch-scope.ts`). Four use cases have dedicated golden-journey suites in `tests/golden-journeys/` ; the others are covered by route, service and integration suites.

USE CASE	IN THE LAUNCH RELEASE	HOW IT REACHES A USER AT LAUNCH
3.1 IND / NDA / BLA authoring	Yes	Authoring (including protocol development) and Submission Center
3.2 510(k) / De Novo	No	In the tree, behind the flag
3.3 CER, EU MDR / IVDR	No	In the tree, behind the flag
3.4 CMC Module 3	Partly	eCTD co-authoring in Submission Center; the dedicated CMC workspace is behind the flag
3.5 CSR	No	In the tree, behind the flag
3.6 Deficiency and CRL response	No dedicated app	AnA is always on, but tools that serve hidden apps are refused in production
3.7 Regulatory intelligence	No	In the tree, behind the flag
3.8 Post-market	No	In the tree, behind the flag
3.9 RIM operations	No	In the tree, behind the flag
QMS controlled documents	Yes	The QMS app (quality, QMP)
Submission readiness	Yes	The deterministic dispatch gate, keyed to the program's sequence

3.1 IND / NDA / BLA authoring (drug). The user opens a program, ingests their nonclinical, CMC, and clinical data, and the platform composes IND Module 2 summaries and Module 3 CMC narratives with sentence-level provenance to source. Golden journey: `tests/golden-journeys/drug-nda-ectd.journey.test.ts` , `ind-authoring.journey.test.ts` .

3.2 510(k) / De Novo authoring (device). Predicate search against the FDA 510(k) registry through the openFDA device client, substantial-equivalence narrative generation, eSTAR-compliant packaging. The `predicate.*` lineage and safety-signal tables are provisioned in schema for a forthcoming local ingest; until then a registry-unavailable state is returned honestly rather than rendered as an empty result. Golden journey: `tests/golden-journeys/device-510k-estar.journey.test.ts` .

3.3 CER for EU MDR / IVDR. Clinical Evaluation Report authoring against MDCG 2020-13, with living evidence spine and post-market update flow. Golden journey: `tests/golden-journeys/cer-eu-mdr.journey.test.ts` . Dedicated IVDR router: `server/routes/ivdr-routes.ts` (84 KB).

3.4 CMC Module 3 co-authoring. Manufacturing narrative generation from process, spec, and stability data — validated by the CMC dataroom review `docs/reports/CMC_MODULE3_DATAROOM_VALIDATION_2026-08-23.md` .

3.5 CSR (Clinical Study Report) generation. ICH E3-compliant CSR builder at `server/services/csr-builder.ts` (853 lines). Draws on `csr_reports` / `csr_details` and the CSR knowledge database at `shared/schema/csr-knowledge-db.ts` (48.9 KB).

3.6 Deficiency and CRL response. The AnA RI orchestrator (`server/routes/ana-ri.ts`) surfaces a curated deficiency taxonomy (static and in-process, `server/services/ana-ri/deficiency-taxonomy.ts`) and a deterministic seven-dimension evaluation rubric exposed at `POST /api/ana-ri/evaluate` for QA scoring. Wiring that rubric inline into the streaming response path is planned, not shipped.

3.7 Regulatory intelligence and horizon scanning. RIM (Regulatory Intelligence Model) at `server/services/intelligence/` ingests guidance, precedent, and reviewer signals; the guidance-impact scanner (`guidance-impact-scanner.ts`) tells you which of your in-flight artifacts a new FDA guidance affects.

3.8 Post-market surveillance (PSUR, DSUR, PMS). `shared/schema/gspr-postmarket.ts` , `capa-mdr.ts` , and the periodic-report pathways in the Intelligent Report Engine (12 report domains, 16 regulatory bodies).

3.9 Regulatory operations at scale. RIM product/registration/label lifecycle (`shared/schema/rim.ts`), submission gateway management (`server/services/submission-gateways/`), CAPA, inspection readiness (`shared/schema/inspection.ts`), IACUC/IBC/IRB (`shared/schema/iacuc.ts` , `ibc.ts` , `irb.ts`).

Most competitors cover one of these. The tree covers all nine on one governance kernel, one audit chain and one design system. The launch release is the narrow slice a first customer needs, validated end to end.

4. The engineered workstreams in the tree

The use cases above map onto nine engineered workstreams, each with its own routes, services, schema domain and tests. Each is a vertical slice rather than a feature. The launch release draws chiefly on workstreams 1, 2, 3, 5, 6 and 8; 4, 7 and 9 stay behind the flag.

- 1. Program & Charter** — `shared/schema/project-charter.ts` (38 KB), `programs.ts` (21 KB). A program has objectives, target markets, target regulators (16 supported bodies plus a multi-regional profile), and a chartered plan.
- 2. Evidence Ingestion & Vault** — `server/services/vault/`. Documents are ingested, chunked (`document-chunking.service.ts`), embedded, and made available for retrieval and lineage.
- 3. Authoring** — `server/routes/authoring.router.ts` (331 KB) and `authoring-actions.ts` (155 KB). Rich-text co-authoring with span-level lineage (`shared/schema/document-span-lineage.ts`): each accepted machine-drafted character range records the machine author that drafted it and every cited source with its checksum at citation time.
- 4. Review & Adversarial Simulation** — `shared/schema/reviewer-simulation.ts` , `shadow-review.ts` . We simulate the regulator on your draft before you submit.
- 5. Governed Actions & Sign-Off** — `server/services/ana-ri/governed-action-signoff.ts` , `part11-governance.ts` . Reason-for-change, TOTP re-auth, signature-integrity check.
- 6. Submission Assembly & Transmission** — `server/services/submission-gateways/governed-transmit.ts` . eCTD/eSTAR packaging and gateway-agnostic transmission.
- 7. RIM & Post-Market Lifecycle** — product/registration/label state machines in `shared/schema/rim.ts` plus the post-market schema.
- 8. Audit, Compliance & Attestation** — `server/services/audit/` (9 services), signed audit exports, chain-integrity sweep job.
- 9. Analytics & Reporting** — the Intelligent Report Engine (3,562 lines, 12 report domains) plus the Submission Twin (1,430 lines).

Each workstream shares four cross-cutting services: **AI Gateway**, **Kernel** (policy + decisions), **Memory** (working/client/project), and **Precedent Engine**. `CLAUDE.md` Rule 2 holds any new workstream until the launch rows are green.

5. System architecture at a glance

At the outermost layer, the runtime is a hardened Express 5 server (Node 22, pinned `>=22.0.0 <23.0.0`) fronting a React 19 client, with a Socket.io real-time channel for collaborative editing and cursor presence, and a Bull-on-Redis queue for long-running AI actions and export jobs. Every browser request lands on a rate-limited, helmeted, tenant-scoped route.

At the next layer, every request that reads or writes governed data flows through:

- **Tenant scope establishment** — `server/middleware/establishRequestTenantScope.ts` opens a Postgres session with the verified JWT's `organizationId` bound, enabling row-level security. `RLS_ENFORCE=on` is fail-closed: any query issued without an active scope is refused.
- **The Kernel** — `server/src/control-plane/kernel.ts` plus eight `server/services/kernel-*` files. Every consequential action is evaluated for governance / security / observability signals and receives a decision (`allow` | `review` | `deny`) with a rationale, a regulatory reference, and a structured evidence payload. Decisions are recorded to `ai_kernel_decision_records` (`db/migrations/20260324_ai_kernel_decision_records.sql`). A tamper-evident hash chain for the kernel decision log exists (`20260325_ana_kernel_log_immutability_hashchain.sql`) but is not yet on the deploy migration set.
- **The AI Gateway** — `server/services/ai-gateway/gateway.ts`. All model calls route through it. Providers, fallbacks, retries, and cost telemetry are handled once, in one place.

Below that sit the intelligence services (RIM, CORTEX Prime, Precedent Engine, Foresight — see §9), the memory services (§10), and the authoring services. Underneath is Neon Postgres with `pgvector` for semantic retrieval and per-row multi-tenant isolation.

The client is a single-page application built with Vite, TanStack Query for server state, and a governed component library. The shell — `client/src/concept2cure/v2/V2App.tsx` with the chrome in `Shell.tsx` (Rail, TopBar, AnaRail, CmdK) — provides the environment; every module is a surface in the catalog at `shared/constants/ui-surface-registry.ts`, and the shell's file-level authority is audited against `config/ui-surface-registry.json`.

6. The control plane: kernel, decisions, and adaptive policy

The kernel is the piece an investor should look at first, because it is the piece competitors cannot casually replicate. A chat wrapper can be rebuilt in a weekend; a policy kernel that has been proven against real submission traffic cannot.

What the kernel actually is. It is a set of pure functions plus a persistence layer that evaluate every governed action against three domains:

- **governance** — is this action allowed under the tenant's regulatory posture and role model?
- **security** — does this action need step-up authentication, and is the requester's context still valid?
- **observability** — is this action being traced, and does the trace carry enough evidence to reconstruct it later?

Each domain returns a `KernelDecision` of `allow` | `review` | `deny` , a rationale string, an ISO timestamp, an optional regulatory reference (e.g. `21 CFR §11.10(e)`), and an evidence payload. The traces compose into a `KernelTraceStep[]` that lives on the request.

The eight kernel services:

FILE	ROLE
<code>kernel-decision-record.ts</code>	Immutable record of every governance decision made for a request
<code>kernel-router.ts</code>	Chooses task type, routing strategy, and token budget for the request
<code>kernel-goal-planner.ts</code>	Generates explicit multi-step <code>GoalPlans</code> with dependencies, success criteria, and replan triggers
<code>kernel-adaptive-policy.ts</code>	Tenant/feature-tier scoped policy bundles that version over time
<code>kernel-observability.ts</code>	Trace and metric emission for kernel decisions
<code>kernel-beta-readiness.ts</code>	Feature-flag readiness gates so we ship the same binary to beta and GA
<code>kernel-plan-runtime.ts</code>	Executes goal plans and reconciles the actual outcome against the planned success criteria
<code>kernel-agent-protocol.ts</code>	The contract agents speak to the kernel over, and the protocol events recorded against it

Why this matters commercially. Regulated buyers cannot ship AI they cannot audit. They need to answer three questions to their own quality organization: *what did the AI do, what said it could, and what will let us prove that in an inspection?* The kernel's decision records are that answer — signed, chained, exportable — and they are the artifact that makes the platform inspectable. Systems without a kernel of this shape will fail the inspection question, whatever else they can demonstrate.

7. The AI Gateway and approved-model governance

The AI Gateway is the governed path for model calls, and a CI guard (`npm run ci:gateway-bypass`) prevents new bypasses from being added. Legacy files that still reach providers directly are tracked in `scripts/ci/gateway-bypass-baseline.json` as an explicit burndown list, each entry stating the governance it loses: nineteen on 2026-09-08, eight on 2026-09-28.

Provider lanes. The gateway routes to seven: `anthropic` , `openai` , `moonshot` (Kimi), `bedrock` , `vertex` , `azure` and `local` (`server/services/ai-gateway/types.ts`). Having lanes is not the product. Which model may answer is.

The approved-models registry. `server/services/ai-governance/approved-models.ts` holds 16 entries. Each carries a pinned version, a written rationale, an evaluation reference and a performance-qualification (PQ) status, and nothing outside the registry is selectable. Approval is enforced at every gateway selection point for drafting and review. Routes and tools that store model-authored content refuse an unapproved serving model, and a refusal leaves an audit trace naming the models withheld. The primary for high-risk authoring and review is Claude Opus 5.5, pinned as `claude-opus-5-5` since 2026-09-25. Opus 5 is retained directly beneath it, so a request the primary cannot serve keeps the reviewed behaviour rather than falling to a smaller model. GPT, Kimi and the local lane ship with their risk tier capped below high-risk until their PQ executes.

Where qualification stands. The PQ is executable (`npm run pq:run`), and a claim that a model passed it is checked against its record. The protocol (PQ-DRAFT-001) is still a draft awaiting owner approval. No registry entry has passed PQ yet, so high-risk drafting in production waits on it. That is a named item under launch row D4 (§19).

Residency. Sensitive dispatch is governed by `AI_PROVIDER_PLACEMENT_APPROVALS` . A tenant's residency or zero-data-retention policy decides which providers it may reach, and the fallback ladder never fails over across that

boundary.

Fallback and retry. Within that boundary, the gateway consults `getFallbackModels()` for the request's task type (nine of them, from `chat` and `document_drafting` through `structured_output` and `embedding`) and walks the ladder. Retries use exponential backoff with jitter ($\text{base delay} \times 2^{\text{attempt}} + 0\text{--}30\% \text{ jitter}$), and non-transient errors (400, 401, 403) abort immediately. A production deploy with no provider configured fails closed rather than serving placeholder text. Canned responses exist for development and tests. Since 2026-09-28 production refuses the explicit opt-in as well — at boot, per request and in the deploy preflight — unless a written acceptance records the risk, and the deploy pipeline refuses that acceptance.

What is measured. `GatewayAuditLogger` persists requests and responses; `GatewayPolicyEngine` enforces policy before dispatch; per-provider health is tracked with `recordSuccess()` / `recordFailure()` so routing can down-weight a struggling provider. Structured logs flow through `createScopedLogger('ai-gateway')`.

Why this matters commercially. A GxP buyer is not buying "several models." They are buying approved-model governance with evidence: a short list of models, each pinned, justified and qualified, and a record of every time a model was withheld. The same discipline runs the other way. Numbers, verdicts and governed content come from deterministic engines, and the model frames, drafts and explains (§18). Because the model narrates rather than decides, an approved model can be substituted without changing an answer.

8. Retrieval: the 9-step provenance-tracked RAG pipeline

The chat surface is not a chat surface. It is a nine-step retrieval and generation pipeline whose intermediate artifacts are auditable. The router is thin (`server/routes/chat.ts`, 99 lines); the pipeline lives in `server/routes/chat/` split across `send-message.ts`, `stream.ts`, `threads.ts`, `upload.ts`, `provenance.ts`, `verifier.ts`, and `shared.ts`.

The nine steps, at high level:

1. **Intent classification** via `IntentLens` — regulatory role and task inference.
2. **Tenant-scoped hybrid retrieval** across the vault (`document-chunking.service.ts`, `document-catalog-search.ts`) using `pgvector` similarity plus keyword blend (`rag-fusion.ts`).
3. **Memory retrieval** — the three-layer memory system fetches working / client / project memory in parallel with per-layer timeouts (`memory-context-assembler.ts`, `memory-orchestrator.ts`).
4. **Precedent injection** — the Precedent Engine returns comparable outcomes and adversarial history for the artifact under discussion.
5. **Context assembly** — `lumen-context-builder.ts` (1,090 lines) composes the system prompt from project context, document context, workflow context, and conversation context, layered onto the static `REGULATORY_SYSTEM_PROMPT`.
6. **Kernel review** — the constructed request is scored by the kernel; low-risk actions proceed, high-risk actions escalate.
7. **Gateway dispatch** — the AI Gateway routes the call to the appropriate model with retry, fallback, and telemetry.
8. **Verification** — `verifier.ts` runs three deterministic claim-quality rules against the retrieved citations: low citation relevance, numbers in the claim absent from every cited snippet, and long claims with thin support. Any rule firing downgrades the claim to WEAK.
9. **Provenance seal** — `send-message.ts` records inputs, outputs, model and decision trace to `ai_retrieval_runs`, with content digests and deterministic JSON canonicalization supplied by `provenance.ts`.

Every one of those steps is testable in isolation and re-runnable on a stored record. That is what lets us claim inspection-readiness, not merely accuracy.

Embeddings. We are pluralist by design. `pgvector` columns exist at three dimensions:

- `1536` — OpenAI `text-embedding-3-small`, the default across the vector-bearing tables (`conversation_working_memory`, `vault.document_chunks`, `precedent.regulatory_precedents`, `cortex.atoms`, `ai.document_embeddings`, and roughly fifteen others).
- `3072` — OpenAI `text-embedding-3-large`, reserved on CORTEX Prime atoms (`cortex.atoms.embedding_3072`) and `ai.document_embeddings`. Precedent records are 1536.
- `1024` — a nullable column on `ai.document_embeddings` reserving Cohere / Voyage / BGE dimensionality. No code path populates it yet: non-OpenAI embedding lineage is schema-ready, not shipped.

An atom-embedding column was declared at 3072 while the writer emitted 1536, so every write was rejected by the column typmod and atom semantic search returned nothing:

`db/migrations/20260730_fix_atom_embedding_dimension.sql` corrected the column to 1536. `cortex.atoms` carries both `embedding_1536` and `embedding_3072` so a tenant could flip embedding models without re-ingesting, though the 3072 column cannot be HNSW-indexed (`pgvector` caps indexed dimensionality at 2,000).

9. The intelligence layer: RIM, CORTEX Prime, Precedent Engine

The intelligence layer is what turns a document store into a submission strategist. It is three services — plus a fourth (Foresight) that we intentionally retired because the shape it took at first was wrong for the domain — and one report engine.

9.1 RIM — the Regulatory Intelligence Model

Path: `server/services/intelligence/` — 37 TypeScript modules.

- **Central orchestrator** `rim.ts` runs at `RIM_VERSION = '1.1.0'`. Every run is stamped with a run ID, the RIM version, and the pattern version, so a decision made six months ago can be exactly re-derived today. Runs that fail to persist are marked `degraded`, not silently swallowed.
- **Judgment framework** `judgment-framework.ts` at `JUDGMENT_FRAMEWORK_VERSION = '1.2.0'` computes six explicit judgment models: Evidence Sufficiency, Defensibility, Reviewer Sensitivity, Claim Risk, Cross-Section Consistency, Submission Risk. Each is a weighted composite of deterministic checks, heuristic checks, and LLM-assisted scoring, with a cross-model `MODEL_WEIGHTS` map (evidence sufficiency 0.25, defensibility 0.20, reviewer sensitivity 0.15, claim risk 0.15, submission risk 0.15, cross-section consistency 0.10).
- **Pattern registry** `pattern-registry.ts` at `PATTERN_REGISTRY_VERSION = '1.3.0'` codifies nine pattern categories (`deficiency`, `reviewer_trigger`, `rejection`, `strong_language`, `weak_language`, `data_gap`, `consistency_issue`, `formatting`, `risk_signal`), with 11 seeded patterns today and room to grow with the corpus.
- Surrounding modules: readiness scoring, recommendation engine, evidence confidence, cross-module intelligence, cross-artifact consistency scanner, learning loop, next-best-action engine, risk model + backtest, precedent mining, outcome-precedent ingestor, guidance impact scanner, calibration, counterfactual replay.

RIM is not a chatbot personality — it is a set of composable scoring functions. Every score has a versioned weight, an evidence trail, and a replay path.

9.2 CORTEX Prime — the knowledge graph

Path: `server/services/cortexPrimeService.ts` — 1,207 lines. Five primitives:

- `CortexAtom` — a single knowledge unit carrying dual embeddings (1536 and 3072) and structured JSONB data. The service's insert path also writes `quality_score`, `metadata` and `source_id`, which no `cortex.atoms` migration defines; the atom write path is not yet schema-aligned.
- `CortexEdge` — typed, weighted, directional relations between atoms.
- `CortexAgent` — a scoped agent identity that reads and writes into the graph.
- `CortexThread` — a conversation that operates over a bounded set of context atoms (`contextAtomIds[]`).
- `CortexTrace` — an audit trail over agent operations on the graph.

CORTEX Prime exposes graph traversal — `startAtomId`, `edgeTypes`, `maxDepth`, `minStrength` — via `cortex.traverse_reasoning`, reached today through the CORTEX REST surface. It is the intended substrate for precedent-aware drafting, cross-artifact consistency across a sponsor's history, and change-impact analysis when a new guidance drops.

9.3 The Precedent Engine

Path: `server/services/precedent-engine.ts` — 2,081 lines. Nine operations:

`precedent.search`, `precedent.compare`, `precedent.risk`, `precedent.strategy`, `authoring.check`, `precedent.crlTriggers`, `precedent.rtfTriggers`, `precedent.emaPatterns`, `precedent.advisoryRisk`.

Backing tables actually queried: `precedent.regulatory_precedents` (the unified corpus), `adversarial.regulatory_adversarial_precedents` (historical FDA questions) and `predicate.predicate_safety_signals`, plus the CSR knowledge tables. The unified corpus is tenant-scoped by construction through `precedent-isolation.ts`, which returns public rows plus the caller's own organization's rows.

What ships today: a sponsor's draft claims can be checked against the precedent corpus (`authoring.check`), and CRL and RTF trigger patterns are scored with calibrated confidence. One caution the launch plan states plainly: the corpus is the platform's memory, and it is empty until ingested. A precedent engine over empty tables is a chat wrapper, so corpus ingestion is launch-critical rather than advisory.

9.4 Foresight — what we removed and why

The foresight subtree — 13 files including `foresight-ai-engine.ts`, three route files and two scripts — was deleted outright; no foresight service remains in the tree. It was retired because it had passed its 2026-04-01 sunset and was surfacing fabricated dose confidence intervals (a flat plus-or-minus 20-25% of the computed dose), and because a general-purpose "predict outcomes" service was the wrong abstraction — it belonged inside the specific judgment models in RIM, tied to specific artifacts and specific regulators, not as a standalone service. Removing it was a correctness decision. The learnings landed in `learning-loop-service.ts` and `outcome-feature-extraction.ts` inside the intelligence layer.

We document what we retire. Retirement decisions are as much a signal to an investor as ship decisions.

10. The three-layer memory system

Everything a competitor does with "remember my chat" we do with a memory system that separates three lifetimes and reconciles them.

Layer 1 — Working memory. `server/services/working-memory.ts`. Structured summaries per conversation with fixed sections: Objective, Locked Facts, Decisions, Open Questions, Next Actions, Created Artifacts. A rolling threshold (`WORKING_MEMORY_THRESHOLD = 20` messages) triggers compaction. Optional semantic search over

working memory via `text-embedding-3-small`, gated by `ENABLE_SEMANTIC_WORKING_MEMORY`. This is what lets a user leave a conversation on Thursday and pick up exactly where they left off on Monday.

Layer 2 — Client intelligence memory. `server/services/client-intelligence-memory.ts`. Sponsor-scoped memory across seven categories — persona, regulatory, pipeline, competitive, operational, preference and history — each entry carrying a title, free-text content, source-document attribution and a confidence score. Every entry has a lifecycle — `status: 'active'` — and can be `superseded` (with a pointer to the successor entry) rather than deleted, so the audit trail is preserved. Retrieval defaults to `includeSuperseded: false`.

Layer 3 — Project intelligence memory. Same store, scoped to a single project or program. Backing tables live in the monolith schema at `projectIntelligenceProfiles`, `projectMemoryEntries`, and `projectIngestedDocuments`. Every project accumulates its own decisions, facts, and open questions — reusable across all users on the project, and across all AI sessions in that project.

The orchestrator. `server/services/memory-orchestrator.ts` coordinates the three layers into a single ranked, deduplicated list. Ranking formula per layer:

$$\text{score} = \text{priorityBoost} + (\text{similarity} \times w_{\text{sim}}) + (\text{confidence} \times w_{\text{conf}}) + \text{verifiedBoost}$$

`memory-context-assembler.ts` (422 lines) reads working memory first, then races the client and project semantic searches in parallel, each under a 3-second timeout with the layer outcome recorded (ok / empty / timeout / error / skipped), and normalizes into the `SharedMemoryContract` — the interface the rest of the platform reads.

Why this matters. In regulated work, "the AI forgets what I told it yesterday" is not a UX complaint — it is a compliance risk. The memory system's supersession lifecycle — entries marked `superseded` with a pointer to the successor rather than deleted — is what lets us claim the AI is a durable collaborator, not a session.

11. Submission Twin and the Intelligent Report Engine

Neither service is in the launch release. Both stay in the tree behind the flag, and they are described here because they show where the architecture goes once launch is done.

Two services that only make sense once the layers above exist:

Submission Twin — `server/services/submission-twin-service.ts`, 1,430 lines. It maintains an in-database model of a regulatory submission — claims, evidence links, drift alerts, reviewer challenges, change impacts and readiness assessments. Analyses run on demand rather than continuously, across six capabilities:

1. **Claim-to-evidence integrity map** — every extracted claim is linked to the artifacts and vault documents that support it, each link carrying a support strength, a relevance score and a staleness signal.
2. **Narrative drift detection** — when the CMC section quietly diverges from the clinical section on the same fact, we catch it before the regulator does.
3. **Regulator challenge simulation** — the twin is prompted through seven reviewer lenses, from skeptical reviewer and evidence-sufficiency skeptic to CMC-heavy, clinical risk and compliance inspection.
4. **Change consequence intelligence** — when an artifact in a submission package changes, the twin identifies the claims and evidence links that referenced it and produces a severity-ranked list of what must be re-verified.
5. **Next best artifact prediction** — given the current state of the twin, what is the single most valuable next thing to produce?
6. **Readiness and fragility modeling** — a submission is only as strong as its weakest section; the twin identifies which sections would collapse under the smallest change.

Tables: `submissionTwinClaims` , `submissionTwinEvidenceLinks` , `submissionTwinDriftAlerts` , `submissionTwinChallenges` , `submissionTwinChangeImpacts` , `submissionTwinAssessments` .

Intelligent Report Engine — `server/services/intelligent-report-engine.ts` , 3,562 lines. Twelve report domains (`regulatory_submission` , `clinical_study` , `cmc_manufacturing` , `pharmacovigilance` , `quality_management` , `compliance_attestation` , `strategic_intelligence` , `provenance_audit` , `device_regulatory` , `biostatistics` , `environmental_safety` , `cross_functional`) targeting sixteen regulatory bodies (FDA, EMA, PMDA, NMPA, TGA, Health Canada, MHRA, ANVISA, MFDS, Swissmedic, ICH, WHO PQ, CDSCO, HSA, SAHPRA, COFEPRIS) plus a multi-regional profile. SHA-256 hash chains, Merkle roots, immutable report records, atom-level report provenance, and indemnification attestations — each intended to answer 21 CFR Part 11 §11.10(a–k).

The engine composes each section from live platform records (projects, CSR data, and `lumen_data_atoms`) against a fixed per-domain section blueprint, recording atom-level provenance for every value it reports. It does not read from the Submission Twin.

12. The authoring surface and governed actions

Authoring is the most-used surface in the product, and it is also the most instrumented.

Where authoring lives. The routes are split across `server/routes/authoring.router.ts` (331 KB, 7,563 lines) and `server/routes/authoring-actions.ts` (155 KB, 3,609 lines). The editor surface on the client is `client/src/concept2cure/v2/surfaces/DocumentAuthoring.tsx` with internals in `client/src/concept2cure/v2/editor/` .

How a governed action works.

1. The user takes an action inside the editor — accept a machine draft, promote a section, sign off on a submission package.
2. The request passes through `authoringObjectAuthorization.ts` : does the requester have access to this artifact, in this project, in this organization?
3. `authoring-actions.ts` resolves the governed context — `resolveGovernedContext()` — validating `projectId` , `artifactId` , `organizationId` , and any escalation gates set by the kernel's adaptive policy.
4. If the action requires an e-signature (approve, submit), `signature-manifestation.ts` composes the human-readable manifestation, `signing-authority.ts` verifies the signer, `mfaService.ts` requires TOTP re-authentication (RFC 6238, HMAC-SHA1, 30s, 6 digits, ±1 window, AES-256-GCM at rest).
5. The signed action is persisted by `server/services/part11/signature-persistence.ts` and written into the hash-chained audit tables by `server/services/audit/chain.ts` , which applies the HMAC seal (`audit-hmac-seal.ts`).
6. Separately, model- and tool-routing decisions are logged best-effort to `ai_kernel_decision_records` for cost and latency analysis. That log is not part of the governed-action audit chain.

Span-level lineage. `shared/schema/document-span-lineage.ts` records, for every accepted machine sentence, the model, the retrieval set, the reviewer, and the timestamp. This is the difference between "the AI wrote this" and "here is the exact provenance of this sentence, and here are the four cited sources."

13. The database: schema, migrations, row-level tenancy

Database work is where the engineering discipline shows up most clearly.

Postgres on Neon, with pgvector. `drizzle.config.ts` targets `postgresql`, reads `DATABASE_URL_ADMIN` / `NEON_DATABASE_URL_ADMIN`, and refuses `.pooler.` URLs during migration — migration DDL must go through a direct Neon endpoint so schema drift cannot happen at the connection-pool layer. The `pgvector` extension is enabled at the SQL level in each migration that needs it.

Migration inventory. 554 migration files total: 247 in `migrations/` (Drizzle-numbered from `0000_sweet_joseph.sql` — a 420 KB baseline — through a dated set spanning `20260327` → `20260908`). Of the 554, 261 are registered in `C2C_MIGRATION_FILES` and re-execute on every deploy; the remainder are historical or laptop-only and are on no applier, and 307 in `db/migrations/` (a GCC-numbered core from `000_gcc_bootstrap_core.sql` → `103_load_complete_templates.sql`, plus its own dated set).

The re-execution discipline. Every migration in the set re-executes on every deploy. This is a deliberate choice, documented in `CLAUDE.md` Rule 1: the applier reads and executes every entry of `C2C_MIGRATION_FILES` unconditionally, and the drift is written to a journal. The rule that follows is a cultural artifact — *"to remove a column, constraint, or table that any file in the set creates, amend the creating migration in place. Do not append a DROP."* The rule is enforced by `npm run ci:migration-drop-safety`, and its failure branch is exercised by its own self-test (`ci:migration-drop-safety:selftest`). Genuine exceptions are baselined with a written reason.

Schema scale. `shared/schema.ts` is an 859 KB monolith with 419 `pgTable(` calls; the `shared/schema/` extraction directory adds another 87 domain files with 275 more `pgTable(` calls (there is some deliberate overlap during migration). Combined ceiling: ~694 tables. The largest domain files: `csr-knowledge-db.ts` (49 KB), `project-charter.ts` (38 KB), `qc-schemas.ts` (30 KB), `orchestration.ts` (26 KB), `operating-system.ts` (26 KB), `capa-mdr.ts` (25 KB), `regulatory-atoms.ts` (23 KB), `programs.ts` (21 KB), `ana-intelligence.ts` (21 KB), `living-record-spine.ts` (20 KB).

Row-level multi-tenancy. `organizationId` appears 736 times in `shared/schema.ts` alone. Most governed tables carry a tenant discriminator — `organization_id` on the newer set, `tenant_id` on older tables such as `audit_logs`. 66 of the 416 `pgTable` definitions in the monolith carry neither; that set is a mix of global reference data and a genuine backlog. The RLS story is layered: `establishRequestTenantScope.ts` opens per-request session scope from the verified JWT; `RLS_ENFORCE=on` is fail-closed; `0021_enable_rls_everywhere.sql` is the canonical enablement migration; `053_gcc_rls_policies.sql`, `069_gcc_multitenant_rls_expansion.sql`, `070_gcc_rls_extended_ga.sql` extend it. Tenant-column audits (`0019_tenant_column_audit.sql`) confirm no unscoped writes.

Audit chain. Eight audit-trail tables in the monolith (`auditLogs`, `auditTrail`, `documentAuditTrail`, `deviceAuditTrail`, `auditEvents`, `proofAuditLogs`, `regulatoryAuditLogs`, `sharepoint_audit_log`), plus HMAC seals and hash chains (`002_gcc_audit_immutability.sql`, `054_gcc_part11_audit.sql`, `064_gcc_cognitive_audit_schema.sql`, `20260609_audit_hmac_seal.sql`, `20260325_ana_kernel_log_immutability_hashchain.sql`), plus signed exports (`server/services/audit/signedAuditExport.ts`) and a chain-integrity sweep job (`server/jobs/auditChainIntegritySweep.ts`).

14. Security, identity, and 21 CFR Part 11 compliance

Security is not a section in the docs — it is a set of gates the code cannot skip.

Identity and authentication. `bcryptjs` for password hashing, `jsonwebtoken` for access tokens. Access token lifetime is 24 hours; refresh token lifetime is 7 days. Failed-login lockout at 5 attempts (`LOCKOUT_THRESHOLD = 5`) for 30 minutes (`LOCKOUT_DURATION_MINUTES = 30`). Every authentication event is logged via `auditService.logAction` with an explicit annotation citing 21 CFR Part 11 §11.10(e).

Rate limiting. `express-rate-limit` (in-process) and a Redis-backed distributed limiter (`redisRateLimiter.ts`). A central rate-limit registry provides seven named tiers — `global` , `api` , `ai` , `auth` , `write` , `upload` , `export` — mounted on `/api/auth` , `/api/ai` , `/api/export` , `/api/upload` , `/api/workflow` , `/api/documents` . Login is 10/15min per IP; signup 5/hr; password-reset 5/hr; MFA-verify 10/15min.

MFA. TOTP is a first-party implementation in `server/services/mfaService.ts` — RFC 6238, HMAC-SHA1, 30-second period, 6-digit code, ± 1 window, 20-byte secret encrypted at rest with AES-256-GCM. No `otplib` dependency. A second, older TOTP path in `server/services/auth-security-service.ts` still uses `speakeasy` and is on the burndown list. Email-OTP fallback (`emailotpService.ts`) and backup codes for recovery.

Enterprise auth. Four-step flow in `server/routes/authEnterprise.ts` : check-email → verify-password → verify-mfa → select-organization. Dev-auth bypasses are removed outright from the enterprise flow; the shortcuts that remain elsewhere are centralized behind `isDevAuthAllowed()` and refuse to run in production. SAML SSO through `server/services/saml-provider.ts` , SCIM 2.0 provisioning through `server/routes/scim.ts` , and a dedicated SSO router.

Transport and headers. `helmet` at two configurations with a CSP nonce (there is a unit test for the CSP-nonce specifically). Content-Security-Policy applied on every response.

21 CFR Part 11. A dedicated `server/services/part11/` folder (`signing-authority.ts` , `reverify-signer.ts` + tests), `server/services/compliance/signature-manifestation.ts` , and the ANA-side counterparts (`server/services/ana-ri/part11-governance.ts` , `governed-action-signoff.ts`). The e-signature path is `governed-transmit.ts` . Signed audit export via `server/services/audit/signedAuditExport.ts` . Nine golden-journey tests run in `tests/golden-journeys/` ; four cover the end-to-end signed submission path per pathway (CER EU-MDR, 510(k) eSTAR, NDA eCTD, IND authoring), the rest covering HAQ correction, marketing application, and submission export, validation and release signature.

GxP, GDPR, HIPAA-adjacent. GDPR: dedicated runtime-role tests (`tests/db/gdpr-service-runtime-role.dbtest.ts`), redaction rules in `sql/redaction_rules.sql` , and tenant-export flows in `server/services/tenant-export/` . GxP referenced through the audit logger, enterprise security, and structured logging. HIPAA-adjacent controls today rest on the audit trail and tenant scoping. A reusable AES-256-GCM field-encryption helper exists (`server/services/security/field-encryption.ts`) but has no production call sites yet; no field is encrypted at rest with it today.

Secrets. `.env` , `.env.*` , and `.env.local` are gitignored (`.gitignore` lines 13-17). Only example files (`.env.example` , `.env.beta.example` , and two other explicit `.env.*.example` files) are tracked. There are no keys in the repo.

Tenancy middleware. `server/middleware/` : `establishRequestTenantScope.ts` (per-request Postgres scope opening), `tenantContext.ts` , `orgMembership.ts` , `authBoundary.ts` , `authAdapter.ts` , `moduleEntitlementGate.ts` , `authoringObjectAuthorization.ts` . RLS is enforced fail-closed, per the design document `docs/architecture/RLS_ROUTE_LAYER_SYSTEMIC_FIX_DESIGN.md` .

15. The frontend surface, design system, and design governance

The frontend is a governed React application, not a design free-for-all.

The shell. `client/src/concept2cure/v2/V2App.tsx` mounts `client/src/concept2cure/v2/Shell.tsx` , which provides four chrome primitives:

- `Rail` — the primary navigation column with up to 24 rail buttons (governed by `railButtonsMax: 24` in `config/ui-surface-registry.json`).

- `TopBar` — organization mark, segment switcher, and the single search entry point that opens CmdK, plus task and help affordances.
- `AnaRail` — the AI copilot rail. Seven surfaces are permitted to take over its column (`ownsConversationMax: 7` , enforced as a layout-flag budget by `scripts/audit-ui-authority.ts`).
- `CmdK` — command-menu launch.

Surfaces. Eleven views ship as first-class surfaces in `config/ui-surface-registry.json` (`Home` , `Chats` , `Projects` , `Project Landing` , `Communication Center` , `Apps` , `Tasks` , `Review` , `Submission Center` , `Editor` , `Vault`), plus two hosted surfaces (`mdx-host` , `pdev-app`) and five destination surfaces.

Governed design tokens. The canonical token source is `design-system/colors_and_type.css` , referenced by the surface registry. Design CI gates enforce no unresolvable token references (`ci:token-cascade`), Lucide-icons-only and no spring or bounce motion (`ci:design-system`), and no *new* shadowed selectors (`ci:check-css-selector-shadowing` , against a declared baseline of 21 pre-existing pairs). Hardcoded hex literals remain a known, documented backlog rather than a blocked condition.

The Ana copilot. Real-time chat surface with three effort modes (standard, deep-research, quick-ask) that map onto the agentic loop's round ceiling (balanced / thorough / fast); the mapping is tested at `client/src/concept2cure/v2/__tests__/anaEffortMode.test.ts` . Image generation is a separate route guarded by `server/middleware/nanoBananaGuard.ts` .

Retired surfaces. The registry also tracks `deleted` surfaces (ZenApp, AnaPersistentPanel, EditorPanel, ProjectWorkspaceShell, GlobalOperatingShell, IndustryWorkspaceShell) so no code path can silently resurrect them. This is a deliberate discipline: we do not just delete UI, we mark it deleted in the source of truth so anyone reading the registry knows the migration is complete.

16. The harness: our engineering discipline as a moat

An investor should read `CLAUDE.md` in the root of the repo before making a decision. It is a short document that sets the engineering culture, and every session — human or AI — works under it.

Rule 0 — one branch. `concept2cure-v2` is the only branch anyone (human, agent, worktree, CI job) may push to. Non-canonical pushes are refused at the pre-push hook. The rule is enforced, and the exception path is narrow (external refs like `dependabot/*` and `revert-*` ; agent-shaped branches are refused unconditionally). This eliminates an entire category of failure — divergent branches, stale mirrors, "which one is truth?" — that consumes the middle years of most startups.

Rule 1 — migrations re-execute. Every migration is re-executed on every deploy, unconditionally. This is a design choice that trades one thing (a small amount of DB work at deploy) for two things (a) an idempotent, replayable schema definition, and (b) a hard rule that DROPs are not the way to remove things — you amend the creating migration in place, and the CI drop-safety gate blocks the alternative.

Rule 2 — no new capability until the launch rows are green. Each session works on one launch workstream, names the row it moves and the evidence it will file, and reports blocked rather than done when it cannot produce that evidence. No session adds a surface, module, tool, model or integration outside the launch catalog. This is the rule that keeps a large tree from turning into a larger one while the first customer waits (§19).

The rules are cultural artifacts codified as enforced gates. `CLAUDE.md` is not a wiki page: it is authoritative, and the hooks cite it. That is how an engineering organization ships quickly without breaking regulated data.

The agents. Alongside the engineers, the repo ships eight specialized reviewer agents in `.claude/agents/` (`ally-auditor` , `design-reviewer` , `design-system-auditor` , `motion-auditor` , `microcopy-reviewer` , `honest-state-auditor` , `part11-ux-auditor` , `security-auditor`), run on demand. The launch plan schedules

the Part 11 UX, honest-state, design-system and security auditors weekly against the launch catalog, with reports filed as periodic-review evidence. Wiring them into `pr-checks.yml` is outstanding. This is how we scale review coverage across 439 client React components without hiring a design-review team.

The doc discipline. 672 markdown files in 55 subdirectories of `docs/` — architecture, audits, runbooks, roadmap, deployment, release, standards, AI governance, compliance, security. Design decisions are written down. Retirements are written down. Trade-offs are written down. An investor is invited to sample five random docs from `docs/architecture/` or `docs/reports/` to verify.

17. Validation, testing, observability, and reliability

The validation package. A computer-software-assurance validation package is drafted and executed for the six launch apps (`docs/validation/` , evidence under `docs/evidence/w3/`). It comprises a validation plan, one user-requirements specification per launch app, a risk assessment, installation qualification, six operational-qualification protocols, a traceability matrix generated from the test runs, and a summary report (VSR-001). The latest run, on 2026-09-27 at commit `89ee3a81` , used an installation provisioned from empty, in the production posture and with production authentication. Every identity except the run identity was created through user administration: each colleague set their own password through the activation link and enrolled their own authenticator.

PROTOCOL	PASS	FAIL	DEVIATION
Installation qualification	12	0	3
Operational qualification (six protocols, 103 steps)	101	0	2
Traceability (TM-001, 71 requirements)	69	0 uncovered	2 partial

Each run has found defects, and each defect is fixed and shown failing first before it is closed. The latest found that a colleague's sign-in events were written outside their organization (F-41), and that the ledger identified an account by display name alone when two accounts shared one (F-42). The package still owes a staging execution with the production image, a PQ-passed model for the drafting step, and signatures from the founder and a qualified contractor (§19, D4).

Automated tests. 2,299 product test files: 1,522 under `server/` , 302 under `client/` , 448 in the dedicated `tests/` tree, 24 under `shared/` and 3 under `scripts/` (a further 53 belong to the agent harness under `.claude/`) that houses integration and end-to-end suites — including `tests/e2e/` (design tokens, submission ops, governed lifecycle, RC-beta path, biotech modules, golden customer journey, beta pulse, submission-ops UI, diff history) and `tests/golden-journeys/` (CER EU-MDR, device 510(k) eSTAR, drug NDA eCTD, IND authoring). Playwright is configured (`playwright.config.ts` , targeting `tests/e2e/`) and runs in the `tier5-browser-smoke` workflow, which installs `@playwright/test` on demand. The `gstack` QA harness (`.claude/skills/gstack/`) runs headless browser dogfooding of live surfaces.

Observability. Sentry (both `@sentry/node` and `@sentry/react`) for error tracking. Structured logging through `logger.ts` . The AI Gateway emits scoped logs (`createScopedLogger( 'ai-gateway' )`) with per-provider health tracking. The kernel persists a decision record per routing decision (`ai_kernel_decision_records` : rationale, selected tools, rejected alternatives, constraints, latency, outcome). The audit chain-integrity sweep runs as a scheduled job (`server/jobs/auditChainIntegritySweep.ts`).

Reliability. Bull-on-Redis queue for long-running AI actions (`refine_with_validation`) and exports (`export_document`) with durable retry (3 attempts, exponential backoff from 2s), failed-job retention for debugging (last 200 within 24h), progress tracking via SSE, horizontal scaling across multiple workers, and graceful drain-on-shutdown. When Redis is unavailable, the queue falls back to synchronous execution rather

than failing the request outright. Socket.io provides the real-time channel for collaboration primitives (`CollaboratorInfo` , `CursorPosition` , `BatchUpdateData`) with JWT authentication.

Deployment posture. Neon Postgres; the admin and provisioning path refuses a pooler host, and the deploy-time applier takes an advisory lock. Migrations replayable. `RLS_ENFORCE=on` fail-closed. `.env` files never in the repo. Feature entitlements resolve per tenant through `server/services/entitlements/require-entitlement.ts` , so the same binary ships to beta and GA.

18. The competitive moat, in one page

Three things are hard to copy, and each is evidenced in the repository rather than asserted.

Engines decide; the model narrates. Numbers, verdicts and governed content come from deterministic engines: the eCTD packager, validators, regulatory clocks, conformance checkers, the substantial-equivalence flowchart, sample-size solvers and rule packs. The model frames, drafts and explains, and a tool that asks a model for a figure is treated as a defect. Only approved, version-pinned models may serve drafting or review (§7). A competitor can wrap a model in an afternoon. Building the engines the model has to defer to is the work.

Part 11 evidence that is executed, not claimed. Every signing path re-verifies the signer through one ceremony (`server/services/part11/reverify-signer.ts`): the account's standing, its lockout, the password and the enrolled second factor. It records what it verified. The audit trail is one HMAC-sealed chain per tenant with an ordered sequence, and the local verifier reads 262 of 262 records OK. An exported signature manifest verifies each printed hash against the content it is printed on, and refuses a sealed record that no signature covers. The validation package — plan, six URS, risk assessment, IQ, six OQ protocols, a traceability matrix generated from the runs, and a summary report — has been executed on a clean install in the production posture (§17).

Tenant isolation proven as the production role. Isolation is tested by a two-tenant contract run as the non-superuser `app_service` role with row-level security enforcing, on an install built from empty. Every cross-tenant path it found was reproduced before its fix and shown holding after: request schemas that took the tenant from the client, child tables with row security off, a membership table with no policy at all, and a vault retrieval pipeline that trusted its caller's organization id. A CI rule now fails on any child table left open.

Underneath all three is a discipline, written into `CLAUDE.md` and enforced by hooks: a gate that has only ever been seen to pass has not been tested. Several of the gates named here — `ci:migration-drop-safety` and `ci:launch-scope` among them — ship with self-tests that show them failing on the case they exist to catch.

The three-layer memory (§10) supports this. AnA's memory is always on, because a user must be able to see what AnA remembers. The Submission Twin (§11) and the Intelligent Report Engine are in the tree but outside the launch release.

19. The launch definition, and where each row stands

This brief carries no roadmap beyond what the repository already commits to.

`docs/LAUNCH_DEFINITION_OF_DONE.md` , binding from 2026-09-20, defines "commercially deployed" as ten rows, each with named evidence filed under `docs/evidence/` . `CLAUDE.md` Rule 2 forbids new capability until every row is green. The state below is taken from that file and the evidence it cites, as of 2026-09-27.

ROW	DONE MEANS	WHERE IT STANDS	STILL OWED
D1 Hosted production	One AWS environment, image promoted by the deploy workflow, /readyz green on schema, AnA, Redis and worker	Application side proven: a database provisioned from empty, one Terraform stack with production and staging roots under test on every push, production refusing to boot without durable document storage	The AWS account, IAM, DNS, secrets and the apply itself; founder decisions on AI placement, Redis, worker and a WAF
D2 Launch catalog	Six apps on by default; every other surface behind a flag off in production; no fixture reachable in production	Green on a local instance with enforcement on: 6 apps, 41 surfaces, 21 modules, and a CI gate with a self-test	The staging run, with D1
D3 Tenant isolation	Runtime as the non-superuser role, RLS_ENFORCE=on, the two-tenant contract passing on staging with the production image	Passing locally as app_service with RLS enforcing, after the fixes listed in §18	The staging run, with D1
D4 Validation package	CSA-aligned plan, URS per launch app, risk assessment, IQ, OQ with executed Playwright evidence, generated traceability, summary report, signed	Drafted and executed locally at one commit: IQ 12 pass / 0 fail / 3 deviations; OQ 101 pass / 0 fail / 2 deviations of 103; traceability 69 of 71 requirements pass, 0 uncovered	Staging execution; a PQ-passed model for the drafting step; signatures by the founder and a qualified contractor
D5 Part 11 evidence	Seal and MFA keys in KMS, a KMS-backed signer, the audit-chain verifier run on production	Signer modes and the KMS envelope implemented against a test KMS; per-tenant chain; local verifier 262/262	The KMS key, one live signed release, the production verifier run
D6 Security posture	SOC 2 Type II window open, third-party pen test closed, questionnaire answered, Anthropic BAA, retention and residency statement	Policy set, questionnaire and trust statement drafted; sign-in defects closed, each reproduced first	SOC 2 platform, pen test, BAA, signatures
D7 One real sequence	One test sequence accepted by FDA's test environment for a real sponsor, with the acknowledgement chain	One AS2 transport for ESG and ICSR; PDF/A output fixed; the regional backbone refuses rather than guesses an application type	Agency DTDs (downloads are refused by the build environment's egress policy), agency accounts, the test submission
D8 Connector for Claude	A remote MCP server with 15–20 curated tools, submitted to the Connectors Directory	Built and proven locally: 19 tools, OAuth 2.1 with PKCE, tenant scoping under RLS, shown failing on a seeded mutation	Staging, a second machine's client, the directory submission
D9 Commercial paper	Pilot, subscription and data-processing agreements, order form, pricing, onboarding and support	Drafts complete under docs/commercial/	Lawyer review; the pricing decision
D10 One customer	A signed, paid pilot with logo rights, and a named regulatory user who has filed a governed document on production	Not yet	Founder

Gates. At week 8, D10 must be signed and D1 staging green; without D10, building stops and the founder spends four weeks only on conversations. At week 16, the pilot user must have filed on production and the D7 sequence must be accepted. If not, the launch date moves and the pilot and any investor are told.

The dashboard. `node scripts/ops/ga-readiness-report.mjs` checks the environment and asset presence behind these rows. The most recent filed run (`docs/evidence/W5/2026-09-24-package-spine-lifecycle/`) reads 5 of 41 ready, with 19 blockers outstanding. Most of the 19 are production secrets and licensed or vendored

agency artifacts: eCTD DTDs, MedDRA and WHODrug, the ESG connection, an agency-grade validator. They are procurement and deployment items rather than code.

Out of scope until launch. The regulatory digital twin, the epistemic, causal and self-evolving engines, federated learning and the manufacturing digital twin stay in the tree behind flags and receive no work until D1–D10 are green.

20. Appendix — quantitative inventory

The numbers below are the ones an engineering diligence reader will want in one place. Every figure is grounded in a file path in the repo.

20.1 Codebase size

DIMENSION	COUNT	REFERENCE
Server TypeScript files	4,180	<code>find server -name '*.ts'</code>
Client React (.tsx) files	439	<code>find client/src -name '*.tsx'</code>
HTTP route modules (top level)	368	<code>server/routes/*.ts</code>
Service files (recursive)	2,956	<code>server/services/**/*.ts</code>
Service subdirectories	222	<code>server/services/*/</code>
Kernel service files	8	<code>server/services/kernel-*.ts</code>
Intelligence layer modules	37	<code>server/services/intelligence/*.ts</code>

20.2 Database

DIMENSION	COUNT	REFERENCE
Migration files total	554	<code>migrations/ (247) + db/migrations/ (307)</code>
Baseline migration size	420 KB	<code>migrations/0000_sweet_joseph.sql</code>
<code>shared/schema.ts</code> size	859 KB	monolith (decimal kB throughout)
Domain schema files	87	<code>shared/schema/</code>
pgTable declarations	~694	419 (monolith) + 275 (domain)
organizationId occurrences	736	<code>shared/schema.ts</code> alone
Embedding dimensions in use	1024, 1536, 3072	pgvector
Report domains supported	12	<code>intelligent-report-engine.ts</code>
Regulatory bodies supported	16 (+ multi_regional)	<code>intelligent-report-engine.ts:56-73</code>

20.3 Versioning of intelligence layer

COMPONENT	VERSION CONSTANT	FILE
RIM	1.1.0	<code>server/services/intelligence/rim.ts</code> line 79
Judgment framework	1.2.0	<code>judgment-framework.ts</code> line 39
Pattern registry	1.3.0	<code>pattern-registry.ts</code> line 26

COMPONENT	VERSION CONSTANT	FILE
Judgment models	6	Evidence Sufficiency, Defensibility, Reviewer Sensitivity, Claim Risk, Cross-Section Consistency, Submission Risk
Pattern categories	9	deficiency, reviewer_trigger, rejection, strong_language, weak_language, data_gap, consistency_issue, formatting, risk_signal

20.4 Security

CONTROL	VALUE	SOURCE
Access token lifetime	24 hours	server/routes/auth.ts:102
Refresh token lifetime	7 days	auth.ts:103
Login lockout threshold	5 failed attempts	auth-security-service.ts:31
Login lockout duration	30 minutes	auth-security-service.ts:32
MFA algorithm	TOTP RFC 6238 / HMAC-SHA1	mfaService.ts
MFA period / digits / window	30s / 6 / ±1	mfaService.ts
MFA secret at rest	AES-256-GCM	mfaService.ts
Enterprise auth steps	4	authEnterprise.ts
SSO / SCIM	SAML + SCIM 2.0	saml-provider.ts, scim.ts
CSP	helmet + nonce	enterprise-security.ts:166,213
Rate-limit tiers	7 named	enterprise-security.ts:417
Audit tables	9 in monolith + more	shared/schema.ts
Audit immutability	HMAC seal + hash chain	audit-hmac-seal.ts, chainIntegrityMonitor.ts
RLS enforcement	fail-closed	RLS_ENFORCE=on

20.5 Tests and docs

DIMENSION		COUNT
Total test files	2,299 product (2,351 incl. 53 agent-harness under .claude/)	
Tests under server/	1,522	
Tests under client/	301	
Dedicated tests/ suite	448	
Golden-journey suites	9 total; 4 cover the signed submission path (CER-EU-MDR, 510(k) eSTAR, NDA eCTD, IND authoring)	
Docs .md files	672	
Docs subdirectories	55	

End of Investor Technical Brief.

Prepared on branch `concept2cure-v2`, the sole production branch, per repository policy.

On 2026-09-08 every factual claim in this document was audited against source: 447 claims checked, 354 confirmed exactly, 91 corrected — including file sizes, one cited path that did not exist, and several capabilities that are scaffolded rather than shipped. Where something is provisioned but not yet wired, this document now says so.

Counts were measured that day and the repository is active; re-run the commands in §20 and expect larger numbers.

Revised 2026-09-28 to align with `docs/LAUNCH_DEFINITION_OF_DONE.md`, which became binding on 2026-09-20. The brief now distinguishes the six-app launch release from the wider tree, frames the gateway as approved-model governance, replaces an earlier roadmap that the repository did not support with the launch rows and their status as of 2026-09-27, and adds the executed validation package. The code counts in §1 and §20 remain those of 2026-09-08.